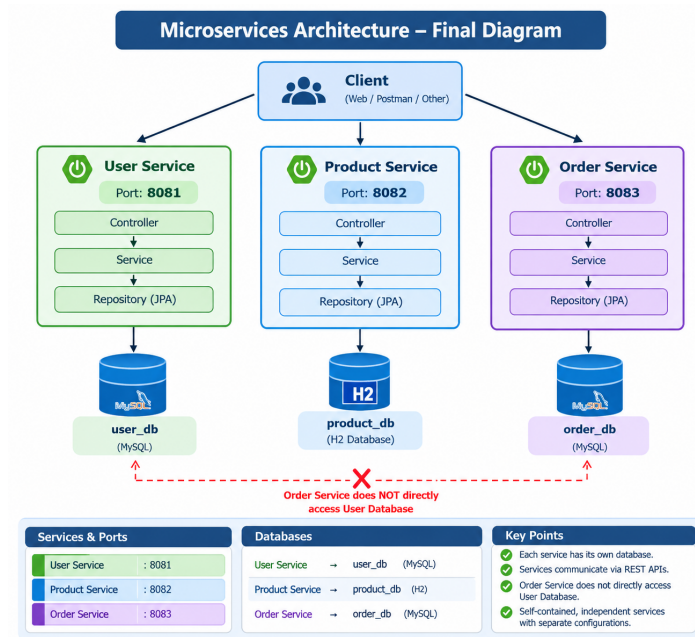


Microservices API Documentation

Spring Boot Microservices Foundation — User, Product and Order Services



1. Service Overview

Service	Port	Responsibility	Persistence
User Service	8081	User CRUD	MySQL: user_db
Product Service	8082	Product CRUD	H2 (current configuration)
Order Service	8083	Order lifecycle	MySQL: order_db

2. User Service API

Method	Endpoint	Purpose
POST	/api/v1/users	Create user
GET	/api/v1/users	Get users
GET	/api/v1/users/{id}	Get user
PUT	/api/v1/users/{id}	Update user
DELETE	/api/v1/users/{id}	Delete user
GET	/actuator/health	Health check

3. Product Service API

Method	Endpoint	Purpose
POST	/api/v1/products	Create product
GET	/api/v1/products	Get products
GET	/api/v1/products/{id}	Get product
PUT	/api/v1/products/{id}	Update product
DELETE	/api/v1/products/{id}	Delete product
GET	/actuator/health	Health check

Note: Product Service was verified to run with the H2 JDBC driver and H2 dialect. Confirm exact controller mappings before creating an automated Postman collection.

4. Order Service API

Method	Endpoint	Purpose
POST	/api/v1/orders	Create order
GET	/api/v1/orders/{id}	Get order
GET	/api/v1/orders/user/{userId}	Get user orders
PUT	/api/v1/orders/{id}/status	Update status
PATCH	/api/v1/orders/{id}/cancel	Cancel order
DELETE	/api/v1/orders/{id}	Delete order
GET	/actuator/health	Health check

5. Order Examples

Create: POST <http://localhost:8083/api/v1/orders>

```
{"userId":1,"productName":"Laptop","quantity":2,"amount":50000,"status":"CREATED"}
```

Update status: PUT <http://localhost:8083/api/v1/orders/1/status>

```
{"status":"CONFIRMED"}
```

Cancel: PATCH <http://localhost:8083/api/v1/orders/1/cancel>

6. Validation / Error Testing

Input / Request	Expected result
{"quantity":2}	400 — userId is required
{"userId":1,"quantity":0}	400 — quantity must be greater than 0
{"userId":1,"quantity":-1}	400 — quantity must be greater than 0
GET /api/v1/orders/9999	404 when order does not exist

7. Database and Service Boundaries

User Service owns user_db. Product Service owns its current H2 persistence. Order Service owns order_db. Order Service must not directly access the User Service database. Services communicate through APIs when inter-service information is required.

8. Health Checks

```
User: http://localhost:8081/actuator/health
Product: http://localhost:8082/actuator/health
Order: http://localhost:8083/actuator/health
```

9. Security Note

Do not commit real database passwords or other secrets to GitHub. Use environment variables or ignored local configuration.